

Function-eval grading report: experiment 08 vs. base under jac 0.36.1

jac-data-gen evals/function/v1, test split (1000 tasks)

Graded 2026-09-16 · analysis of `fn_eval_preds/`

Executive summary

- **The LoRA (08) works.** Same 1000 tasks: the adapter lifts pass@1 from **0/1000** (base) to **219 passes**, and compile rate from 5.9% to **73.7%**. It also taught contract discipline: 99.7% of 08 outputs stay inside the feature contract vs. 59.2% for base.
- **But the 08 grade is not trustworthy as printed.** The grading machine ran out of disk mid-run: 251 tasks are marked `infra_error` (grader exit 2, run incomplete), and another **59 “test_fail” rows are really “postgres failed to start”** infra errors that the grader’s infra regex missed. Corrected: **310 of 1000 tasks (31%)** were infra-killed, not model failures.
- **On cleanly-graded tasks 08 scores 31.7% pass@1** (219/690). Projecting the infra tasks at the same compile-conditional rates gives an estimated **33–37%** for a complete run — i.e. the reported 29.2% understates the model.
- **Translation is the adapter’s strong track** (50.2% pass, 78.7% compile); completion is weak (15.0% pass, 46.5% compile), mostly because the model truncates or malforms the provided prefix (75% of completion check-fails).
- **jac 0.36.1 grading itself is non-deterministic.** Re-grading 12 sample tasks on a healthy disk flipped 2 of 2 pass-controls to test_fail — the same code passes via the *native* path but fails via the *server codespace* path (e.g. E1053 type errors that only fire in the server runtime). All 3 check-fail controls reproduced exactly. Passes are lucky rolls; treat every pass@1 from this toolchain as an upper bound.
- **Base fails in a characteristic way:** 0/500 compilable translations — it writes *old-Jac syntax* (`func f(..): T`) that the 0.36.1 parser rejects (72% of all its check-fails), plus degenerate repetition loops that break extraction on 124 completion tasks.

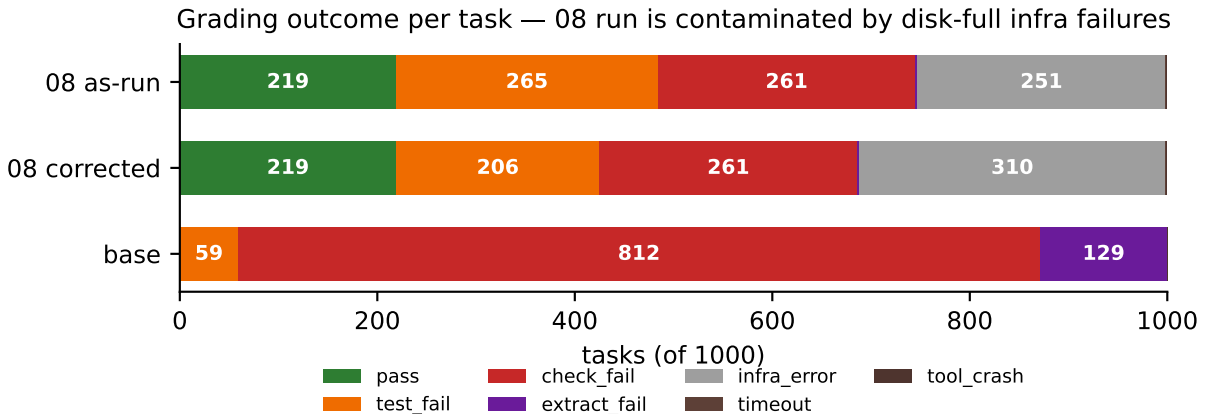


Figure 1: Outcomes for all 1000 tasks. “08 corrected” reclassifies 59 postgres-startup failures from `test_fail` to `infra_error`. Base’s 9 hidden infra failures are shown in the base bar too (see §4).

1 Setup

Two systems answered the same 1000 tasks (500 *completion*: finish a partly written Jac function from a prefix; 500 *translation*: port a Python function to Jac). Each task hides Jac `test` blocks. A task **passes** only if the candidate

1. yields a parseable `““jac` block (else `extract_fail`),
 2. survives `jac check` (else `check_fail`),
 3. passes every hidden test under `jac test` (else `test_fail`), and
 4. keeps the feature contract (no Python syntax, no `import:py, ...`).
- **08** = Qwen3-Coder-30B-A3B (MLX 4-bit) + experiment-08 LoRA adapter.
 - **base** = same model, no adapter.
 - Grader: `eval_jac.py` (jac-data-gen @ 5222202a), 4 threads × 2 pytest workers, 900s per stage, **jac 0.36.1** — the toolchain the eval targets. The original experiment grade used 0.16.1.

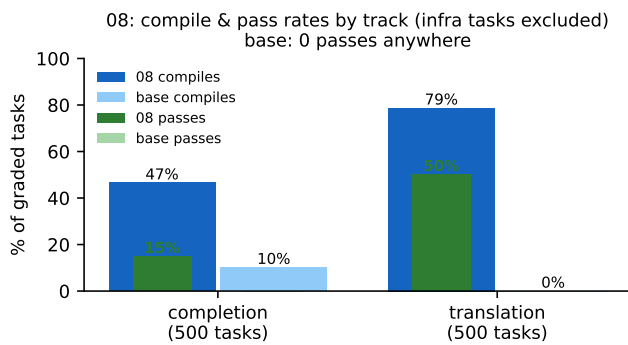
2 Headline numbers

	pass@1	compile	contract	run	note
08, as reported	29.2%	73.7%	99.7%	incomplete	251 infra; pass/749 eligible
08, corrected	31.7%	—	—	graded 690/1000	59 pg-fails moved to infra
08, projected full	~33–37%	—	—	estimate	see §4
base	0.0%	5.9%	59.2%	complete	0/1000 pass

Reference: same preds under jac 0.16.1 (from README): 08 pass@1 57.0%, base 3.0%.

Table 1: Overall results. The 0.16.1 comparison is indicative only: that run used a different parser/type-checker, and this run’s 08 grade is contaminated (see §4).

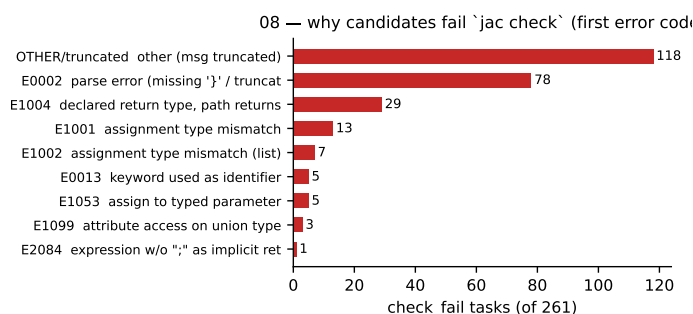
3 Task breakdown



08 (infra excluded)	compl.	transl.
graded	361	329
pass	54	165
	15.0%	50.2%
compiles	168	259
	46.5%	78.7%
check_fail	191	70
test_fail	112	94
extract_fail	2	0

Figure 2: 08 by track. Translation — the adapter’s main gain: 3.3× the completion pass rate. Completion fails early: 191/361 never even parse.

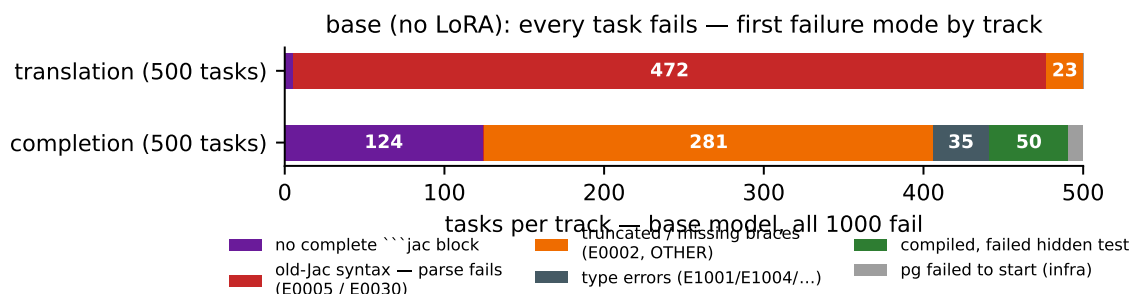
Where 08's check-fails come from



Completion check-fails (191) are dominated by **structural damage to the given prefix**: missing `}` (E0002, 76) and truncated/mangled output (88 whose message tail is cut before the code appears). Translation check-fails (70) are **semantic**: the 0.36.1 type checker rejects assignments and return paths that 0.16.1 accepted (E1001/E1002/E1053/E1099/E1004).

Figure 3: First error code per 08 check-fail.

Base: failure anatomy (all 1000 tasks fail)



Base never emits valid 0.36.1 Jac for translation: 472/500 die in the parser because it writes pre-0.4 “old Jac” signatures, e.g. for `fn-translate-337455`:

```
func _cmp(kd1: any, kd2: any): int { # old syntax: colon return type
  ...
}
# -> E0005 Unexpected token 'return' ... E0030 Unexpected semicolon at module level
```

On completion tasks it loops: 379/1000 base responses contain degenerate line repetition; when the loop swallows the closing fence the extractor gives up (124 `extract_fail`):

```
```jac
 if k == 'registry' {
 return 'registry'
 }
 if k == 'registry' { # ... repeated 40+ times, fence never closed
```

The 50 base completions that do compile still fail their hidden tests — base’s 3.0% pass@1 under 0.16.1 was already luck-level; under the stricter 0.36.1 checker it is 0.

## What the LoRA actually taught

### 4 The 08 run is contaminated: infra forensics

1. **Disk filled up during the run.** The first infra error appears at task  $\approx 267$ ; from then on the run alternates between working windows and windows where jac’s embedded PostgreSQL cannot write (`PgWireError: ...No space left on device`) or cannot start at all. All 310 infra tasks had already passed `jac check` and died in the test stage.

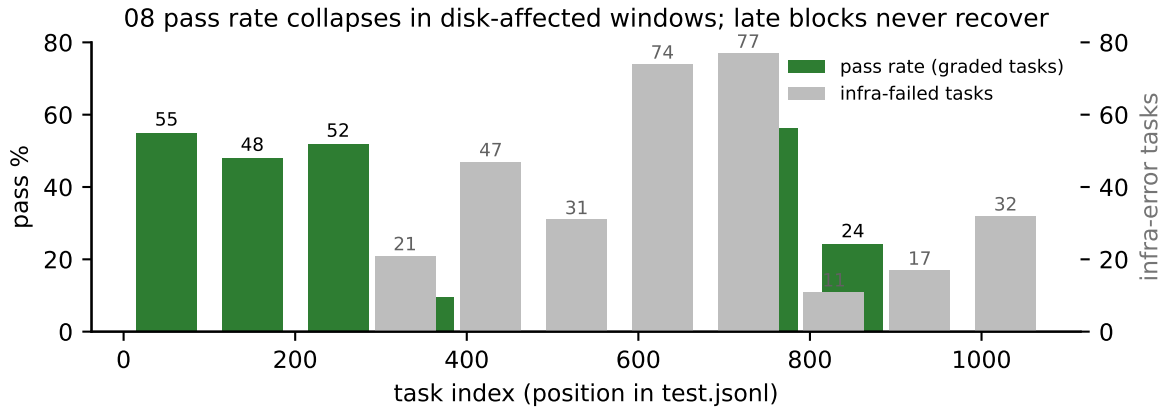
2. **The grader undercounts infra failures.** Its infra regex does not know the `AssertionError: postgres failed to start: pg_ctl: could not start server collection error`, so 59 of those were recorded as ordinary `test_fail` (0 passes, 94 tasks graded in block 900+ — all failures, zero passes). Base’s run, on a

output statistic (1000 preds)	base	08
opens translation fence with ‘‘jac	99.0%	99.8%
typed signatures ( <code>typed.def</code> )	43.2%	99.3%
no Python-syntax smell	68.1%	99.9%
inside feature contract	59.2%	99.7%
new-style <code>def f(..) -&gt; T { }</code> on translation	~0%	~100%
repetition-loop responses	379	333

**Table 2:** The adapter’s job was mostly *syntax and format normalization*: new-Jac signatures, static types, no Python leakage. Base simply speaks the wrong dialect.

healthy disk, still hit 9 of the same pg-start failures — the pg startup is flaky under 4-way concurrency even with space free.

**3. A stale postmaster is still running.** A PostgreSQL started at 17:42 during the run (PID 233078, socket `/tmp/jacpg-f4480dba16`) is alive and a stale lock in `/tmp/jacpg-4f008fa8cc` blocks new `jac test` runs until cleaned.



**Figure 4:** Pass rate per 100-task block (green, left axis) vs. infra-killed tasks (grey, right axis). Windows with heavy infra damage score  $\approx 0$ ; clean windows recover to 50–56% — evidence the collapse is environmental, not model quality.

## Projected score for a complete, clean run

Infra tasks all compiled. Applying the clean compile-conditional pass rates (completion  $54/168 = 32.1\%$ , translation  $165/259 = 63.7\%$ ) to the 139 completion + 171 translation infra tasks projects  $\approx 99 + 274 \approx$  **373 passes (37%)**. The conservative version (infra tasks at the unconditional graded rates) gives  $\approx 326$  (33%). Honest summary: **08’s true score is between the corrected 31.7% and  $\approx 37\%$ , not the printed 29.2%.**

## 5 The grader is non-deterministic: re-run experiment

After freeing disk (35 GB free), I re-graded 12 tasks (5 controls + 7 suspicious failures) with the same binary, grader, and flags (`PYTEST_XDIST_AUTO_NUM_WORKERS=2` had no effect on the outcome):

original status	re-grade status	count
check_fail	check_fail (identical)	3/3
pass	test_fail	2/2 flipped
test_fail	test_fail	7/7 (but see below)

**Table 3:** Stability on re-grade.

The two flipped tasks are the smoking gun. `fn-translate-337455` passed originally and failed on re-grade with `error[E1053]: Cannot assign dict[int, int] to parameter 'kd1' of type int | str` — its code annotates `kd1: int | str` but a hidden test calls `_cmp({2:1}, {2:1})`. The other flipped on `error[E5092]: Native lowering failed for builtin call 'min'`. Both messages carry the note “*preferred native but did*

*not lower; compiled in the server codespace*”: jac 0.36.1 runs tests through two different engines (native lowering vs. the PostgreSQL-backed server runtime), they disagree on type strictness, and which engine executes a given task varies run to run. **Consequences:** check\_fail verdicts are trustworthy; individual passes are not reproducible; any pass@1 measured with this toolchain is an upper bound on true functional correctness.

## 6 Recommendations

1. **Re-run 08 end-to-end** on a machine with headroom; keep an eye on df mid-run, or cap concurrency (WORKERS=2) to reduce pg pressure.
2. **Patch the grader’s infra regex** to also match postgres failed to start / pg\_ctl / PgWireError so pg startup failures stop masquerading as model failures.
3. **Before trusting passes, re-grade them once** and count only pass/pass (or report best-of-2 and both-of-2). For training-signal purposes, treat reward from this grader as noisy-positive.
4. **Clean up before the next run:** kill stale postmasters and remove /tmp/jacpg-\* lock dirs.
5. **Data-side lever for completion:** the dominant completion failure is structural damage to the prefix (E0002/truncation, 164 of 361 graded). More training pairs that *copy the prefix verbatim and close all braces* is the cheapest win; translation is already at 50% and mostly limited by the 0.36.1 type checker (strictness 0.16.1 never applied).

## Appendix: artifacts

path	content
out/{08,base}/results.jsonl	one row per task: status, error, static stats
out/{08,base}/summary.json	grader aggregate
out/{08,base}/grade.log	grader console output
report/stats.json	every number in this report
report/c*.pdf	figures
/tmp/regrade*/results.jsonl	§5 re-run evidence